

Mantenimiento predictivo en la industria

Trabajo práctico nro. 1



Gonzalo Gaitán, Marcos Debona, Gonzalo Agüero, Franco Ocampo

Facultad regional Santa Fe, Universidad Tecnológica Nacional

Inteligencia Artificial, Ingeniería en Sistemas de Información

Mariel Ale; Mauro Pacchiotti

21 de mayo de 2026

Índice

Índice.....	1
Introducción	2
Análisis Exploratorio de Datos	3
Forma de los Datos	3
Clasificación Final de las Columnas del Conjunto de Datos	4
Medidas de Tendencia Central y Distribución de las Variables	5
Medidas de Dispersión de las Variables	6
Relaciones Entre los Datos	6
Proporciones de las Columnas Categóricas	6
Detección de Valores Atípicos o Incorrectos.....	8
Relación Entre Variables	15
Análisis de correlación.....	15
Preprocesamiento de los datos	18
Eliminación de columnas irrelevantes	18
Codificación de variables categóricas.....	19
División del conjunto de datos.....	19
Tratamiento de valores inválidos y faltantes	19
Normalización.....	21
Balance del conjunto de datos.....	22
Entrenamiento y evaluación de modelos	22
Metodología común	22
KNN.....	23
Naive Bayes	23
Árbol de Decisión	24
Random Forest (modelo ensamblado)	25
Comparación de Modelos	27

Curva ROC y AUC	28
Tiempos de cómputo	30
Conclusión	32

Introducción

Una fábrica brinda información recolectada de sus máquinas que describe el estado de la herramienta en uso y las condiciones del proceso productivo. Variables como la temperatura de la herramienta, la temperatura del ambiente, la velocidad de rotación, el torque y el desgaste de la herramienta, combinadas con el registro histórico de si en esas condiciones se produjo una falla o no, permiten entrenar un modelo de aprendizaje automático capaz de predecir cuándo una máquina está próxima a fallar. Esto habilita a la empresa a anticipar el mantenimiento necesario, reduciendo los cortes no planificados de producción y mejorando la productividad general. A lo largo de este trabajo se analizan y preprocesan los datos disponibles, con el objetivo de entrenar diferentes modelos de aprendizaje automático y evaluar su desempeño al realizar esta tarea de clasificación binaria.

Análisis Exploratorio de Datos

Esta sección detalla el Análisis Exploratorio de Datos (EDA), implementado con el propósito de caracterizar la estructura del conjunto de datos, identificar patrones subyacentes, detectar anomalías y diagnosticar la calidad de la información disponible. El análisis se fundamentó en el cálculo de descriptores matemáticos y estadísticos que facilitaron la interpretación rigurosa de las variables. Con el fin de garantizar la claridad y la reproducibilidad de los resultados, los hallazgos se estructuran analíticamente a lo largo del informe mediante matrices de datos y soporte gráfico.

Forma de los Datos

Para la comprensión de los datos, fue necesario analizarlos desde diferentes perspectivas. En la siguiente tabla se muestran 5 filas del conjunto de datos, a fin de dar un pequeño vistazo y analizar “como lucen lo datos”.

idx	parent_device_id	product_type	air_temp [K]	process_temp [K]	speed [RPM]	torque [Nm]	tool_wear [min]	target
0	0	5291	L	303.80	313.30	1406	52.10	166 normal
1	1	3908	L	302.30	311.20	1633	36.40	114 normal
2	2	10767	M	302.27	311.47	1228	68.75	187 failure
3	3	15484	L	302.35	311.93	1522	36.99	214 failure
4	4	784	L	296.70	307.90	1743	27.90	55 normal

Tabla 1: Primeras 5 filas del conjunto de datos.

Del análisis del archivo del conjunto de datos (*i40 (datos).csv*) y la visualización de la tabla 1, pudimos observar que el conjunto de datos contiene 14521 filas y 9 columnas. Además, pudimos inferir que probablemente tanto la columna *idx* como *parent_device_id* no aportan

información relevante para el análisis, ya que no representan más que identificadores para cada fila, por lo que decidimos descartarlos para simplificar el trabajo.

La siguiente tabla (tabla 2) muestra por cada columna la cantidad de valores “no nulos” que existen y el tipo de cada una:

#	Column	Non-Null Count	Dtype
0	idx	14521 non-null	int64
1	parent_device_id	14521 non-null	int64
2	product_type	14521 non-null	object
3	air_temp [K]	14481 non-null	float64
4	process_temp [K]	14521 non-null	float64
5	speed [RPM]	14521 non-null	int64
6	torque [Nm]	14521 non-null	float64
7	tool_wear [min]	14521 non-null	int64
8	target	14521 non-null	object

Tabla 2: Información general de cada columna.

A partir de esta tabla se pudo observar que casi todas las columnas tienen 14521 valores no nulos, lo cual es lo esperado considerando que el total de filas del conjunto de datos es de esa misma cantidad. Por otro lado, para la columna *air_temp [K]* se puede observar que solo hay 14481 filas con valores distintos a nulos, lo que indica que hay al menos 40 filas con la columna *air_temp [K]* faltante. Esto es importante, ya que luego de esta observación se tuvo que considerar la reparación de estos datos faltantes mediante técnicas de imputación, como *KNNImputer*. Se pudo observar, además, que las únicas columnas categóricas relevantes son *product_type* (tipo de producto) y *target* (columna a predecir).

Clasificación Final de las Columnas del Conjunto de Datos

Luego del análisis de los tipos de datos de cada columna y lo que representan, determinamos que la siguiente son las clasificaciones para cada una:

- *idx*: Categórico Nominal
- *parent_device_id*: Categórico Nominal
- *product_type*: Categórico Nominal
- *air_temp [K]*: Numérico Continuo
- *process_temp [K]*: Numérico Continuo
- *speed [RPM]*: Numérico Discreto
- *torque [Nm]*: Numérico Continuo

- *tool_wear [min]*: Numérico Discreto
- *target*: Categórico Nominal

Aclaración: Respecto a la variable *product_type*, si bien se ha catalogado preliminarmente como categórica nominal, tenemos la sospecha de que podría exhibir un orden jerárquico subyacente (categórica ordinal). La validación de esta naturaleza ordinal se posterga para la etapa de análisis profundo y codificación de variables (*feature engineering*), sección en la cual se expondrán las pruebas empíricas y se tomará la determinación definitiva.

Medidas de Tendencia Central y Distribución de las Variables

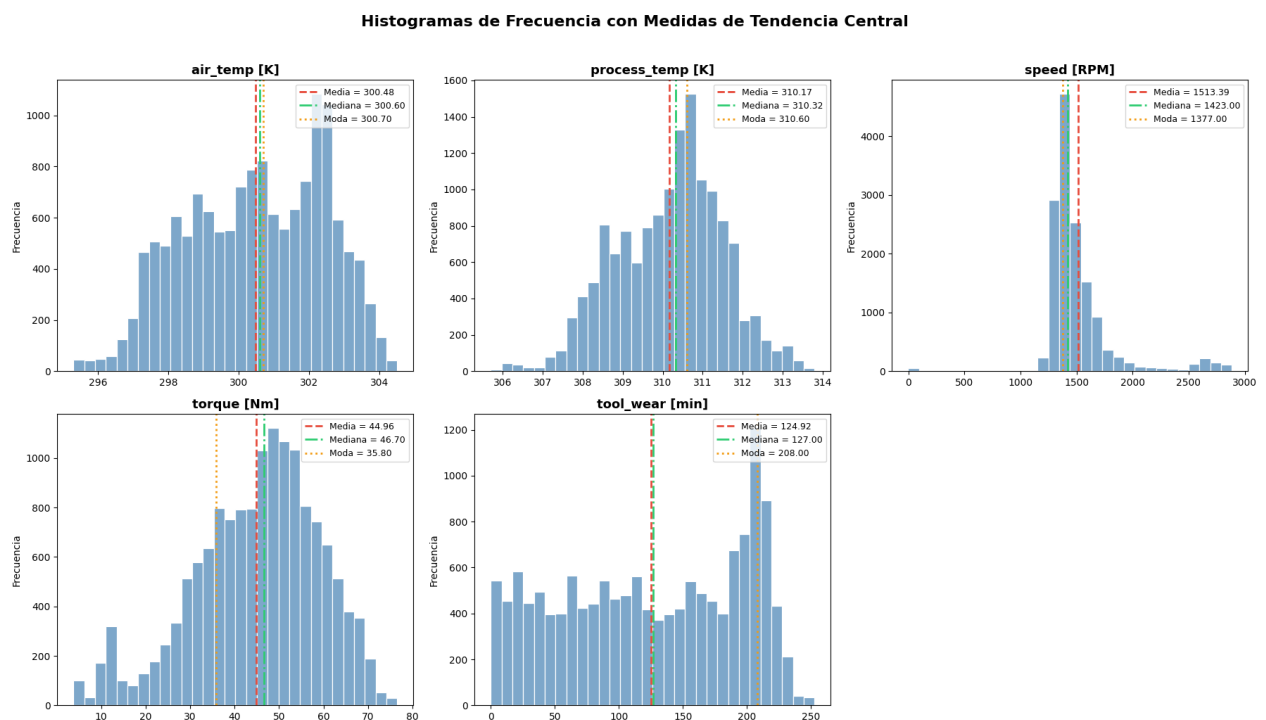


Figura 1: Histogramas de columnas numéricas con indicadores de la media, mediana y moda.

En la figura 1 se puede observar la forma de los datos de las columnas numéricas con respecto a sus distribuciones de probabilidad. Este gráfico también provee líneas punteadas de distintos colores para indicar la media (rojo), mediana (verde) y moda (naranja) de cada columna.

Las siguientes son observaciones realizadas a partir del análisis de la figura 1:

- Las columnas *air_temp [K]*, *process_temp [K]* y *torque [Nm]* presentan distribuciones aproximadamente normales.

- Las columnas *speed [RPM]* y *tool_wear [min]* siguen una distribución distinta. Específicamente, se puede intuir que *speed [RPM]* sigue una distribución exponencial.

Medidas de Dispersión de las Variables

	Mínimo	Máximo	Rango	Q1	Q3	IQR	Varianza	Desv. Est.	CV (%)
Variable									
air_temp [K]	295.3	304.5	9.2	298.90	302.20	3.30	3.81	1.95	0.65
process_temp [K]	305.7	313.8	8.1	309.20	311.09	1.89	1.77	1.33	0.43
speed [RPM]	-1.0	2886.0	2887.0	1354.00	1561.00	207.00	98347.58	313.60	20.72
torque [Nm]	3.8	76.6	72.8	35.99	55.00	19.01	202.40	14.23	31.65
tool_wear [min]	0.0	253.0	253.0	64.00	193.00	129.00	4914.85	70.11	56.12

Tabla 3: Medidas de dispersión de los datos por columna.

Relaciones Entre los Datos

Proporciones de las Columnas Categóricas

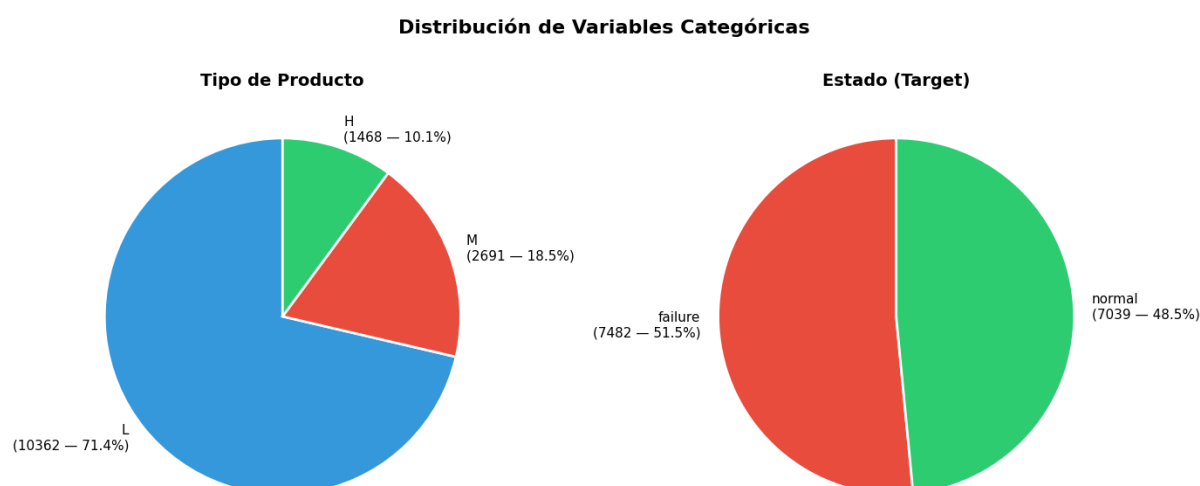


Figura 2: Proporción de Variables Categóricas

A partir de las distribuciones presentadas en la Figura 2, se pueden identificar el comportamiento y la representatividad de las variables categóricas del dataset. En primer lugar, el gráfico de Tipo de Producto (*product_type*) exhibe un claro predominio de la categoría L (71.4%), seguida por la M (18.5%) y la H (10.1%). Si bien las etiquetas no se encuentran explícitamente definidas en la documentación inicial, la distribución decreciente de sus frecuencias permite teorizar que reflejan algún tipo de segmentación jerárquica o estándar de producción (como niveles de calidad o criticidad), lo que aportaría un indicio visual clave para sostener la hipótesis de una relación ordinal entre estas variables.

Por otro lado, el análisis de la misma figura revela un equilibrio casi perfecto en la variable objetivo (target), donde las fallas (failure) representan el 51.5% y el estado normal el 48.5%. Desde una perspectiva industrial, este escenario es altamente atípico, ya que en entornos de producción reales las fallas mecánicas son eventos raros o de baja frecuencia (comúnmente inferiores al 5%). Por lo tanto, se teoriza que el conjunto de datos original ha sido sometido previamente a un proceso de balanceo artificial (como una técnica de sobremuestreo de la clase minoritaria o un submuestreo de la mayoritaria). Dado que el dataset ya se presenta en este estado óptimo y equilibrado, se descarta la necesidad de aplicar balanceos adicionales en nuestra etapa de preprocesamiento, procediendo directamente al entrenamiento de los modelos bajo un escenario ideal de paridad.

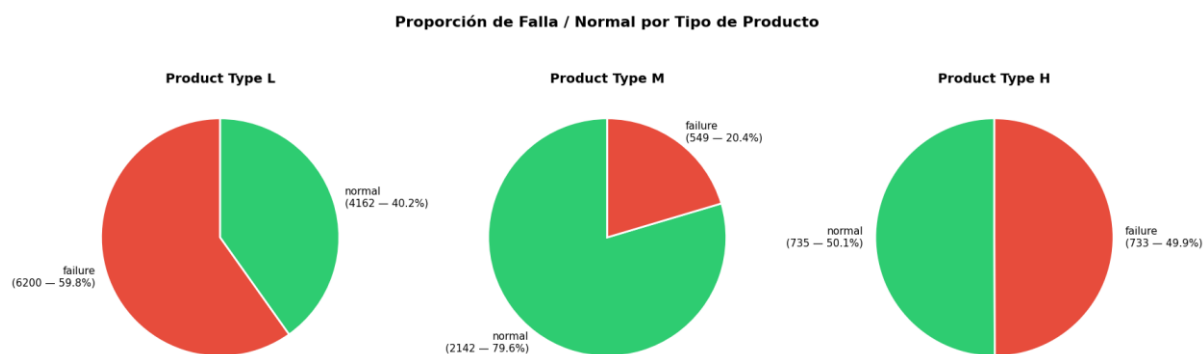


Figura 3: Proporción de Falla/Normal por Tipo de Producto.

Observaciones

El análisis estadístico presentado en la Figura 3 permite profundizar en la interacción entre `product_type` y la variable objetivo (target). Los resultados evidencian que el comportamiento de falla no es homogéneo entre las categorías. Por un lado, la variante L se posiciona como la más susceptible, con un 59.8% de registros defectuosos, lo que denota una marcada debilidad estructural o un proceso operativo más exigente para este tipo de producto. En el extremo opuesto, la variante M se consolida como la más estable, limitando sus fallas a un 20.4%.

Resulta particularmente llamativo el escenario de la variante H, la cual presenta una división prácticamente simétrica (49.9% failure vs. 50.1% normal), esta distribución equitativa robustece la teoría de un remuestreo artificial previo en el dataset para homogeneizar las clases. En síntesis, la marcada disparidad en las tasas de fallo entre L, M y H valida empíricamente la

hipótesis de que `product_type` no es una variable puramente nominal, sino que arrastra un peso jerárquico u ordinal que impactará directamente en la capacidad predictiva de los algoritmos.

Detección de Valores Atípicos o Incorrectos

Para la detección de valores atípicos se utilizó el método del rango intercuartílico (IQR), considerando como outliers aquellos valores por fuera del intervalo $[Q1 - 1,5 \cdot IQR ; Q3 + 1,5 \cdot IQR]$. Este criterio se aplicó sobre cada variable numérica de forma individual.

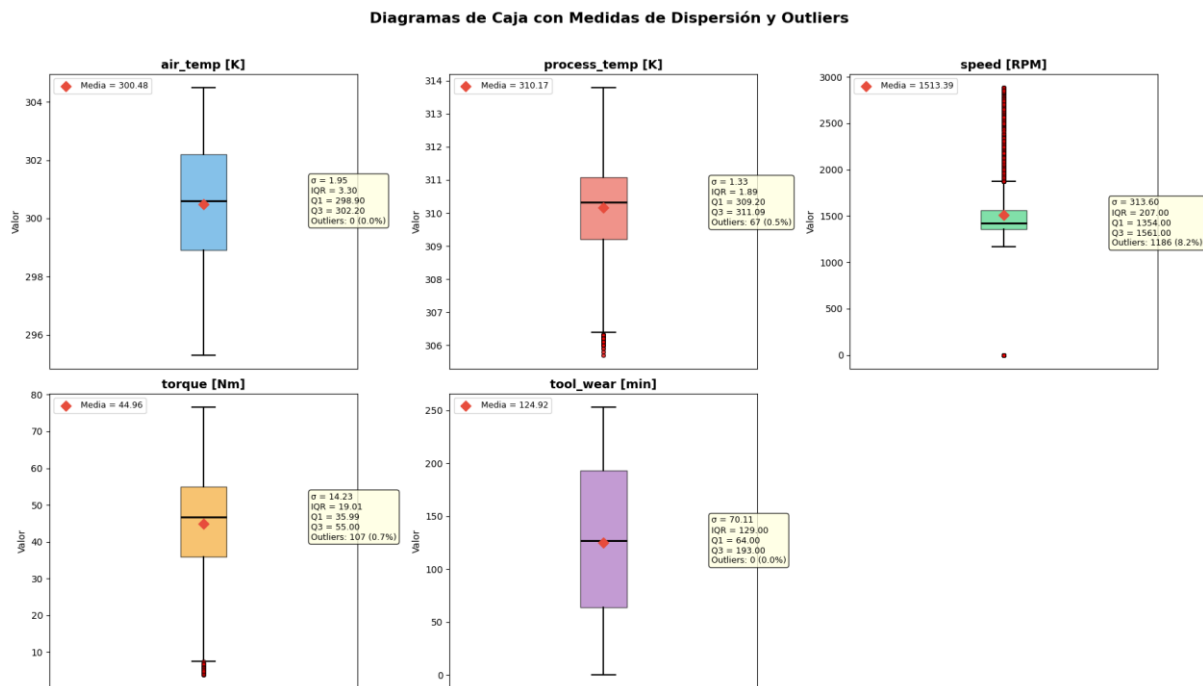


Figura 4: Diagramas de Caja con Medidas de Dispersión y Valores Atípicos.

Mediante el análisis realizado para la figura 4, pudimos observar varios valores atípicos en las columnas `process_temp [K]`, `speed [RPM]` y `torque [Nm]`. Sin embargo, analizando detalladamente logramos notar que la gran mayoría de estos valores atípicos no son valores incorrectos, ya que sí bien son valores más raros, son al mismo tiempo totalmente posibles y válidos. Por ejemplo, un valor de `process_temp [K]` de 306 K no es algo descabellado si consideramos que la media es de 310.17 K.

Por otro lado, para el caso de `speed [RPM]`, sí pudimos detectar valores inválidos. Como se puede observar en la Figura 4, hay valores de velocidad cercanos a cero. Haciendo una exploración de la columna para este rango de valores, se identificó la presencia de 47 filas con `speed [RPM] = -1`, lo que es físicamente imposible. Este valor puede ser un indicador de valor faltante o un error de medición de los sensores. Cualquiera sea el caso, luego de esta observación se decidió que el mejor enfoque sería imputar estos valores.

Impacto de los Valores Atípicos de Velocidad en las Fallas

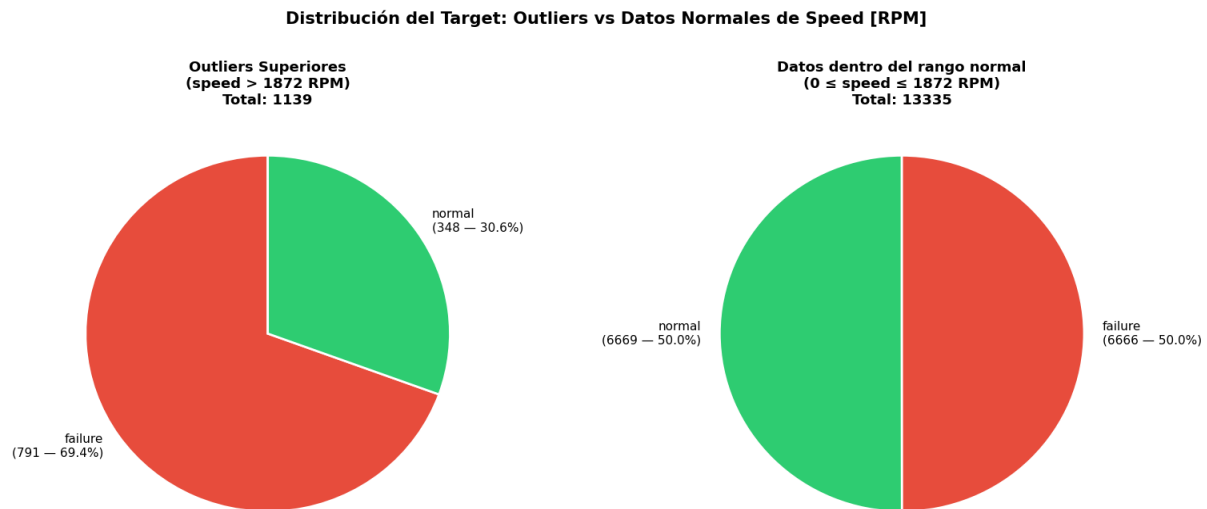


Figura 5: Distribución del target para Valores Normales vs Outliers en Speed.

Con el propósito de determinar si los valores extremos de la variable Speed contienen un patrón predictivo útil, se contrastó la distribución del Target entre el subconjunto de *outliers* superiores y los datos del rango normal (Figura 4). El análisis revela que, al superar las 1872 RPM, la proporción de fallas asciende al **69.4%**, marcando un contraste evidente con la paridad exacta (50/50) observada en el remanente de los datos operacionales. Esta asimetría confirma que las velocidades extremas no constituyen ruido aleatorio, sino que actúan como un indicador crítico de riesgo operativo. Esto se debe a que operar por encima de los límites de diseño somete al equipo a condiciones para las que no fue diseñado: las fuerzas sobre las piezas en movimiento se multiplican, la fricción genera calor en exceso y la carga mecánica acumulada deriva en fallas graves o interrupciones no programadas.

Distribución del Target por Tipo de Producto en Rangos de Velocidad Críticos

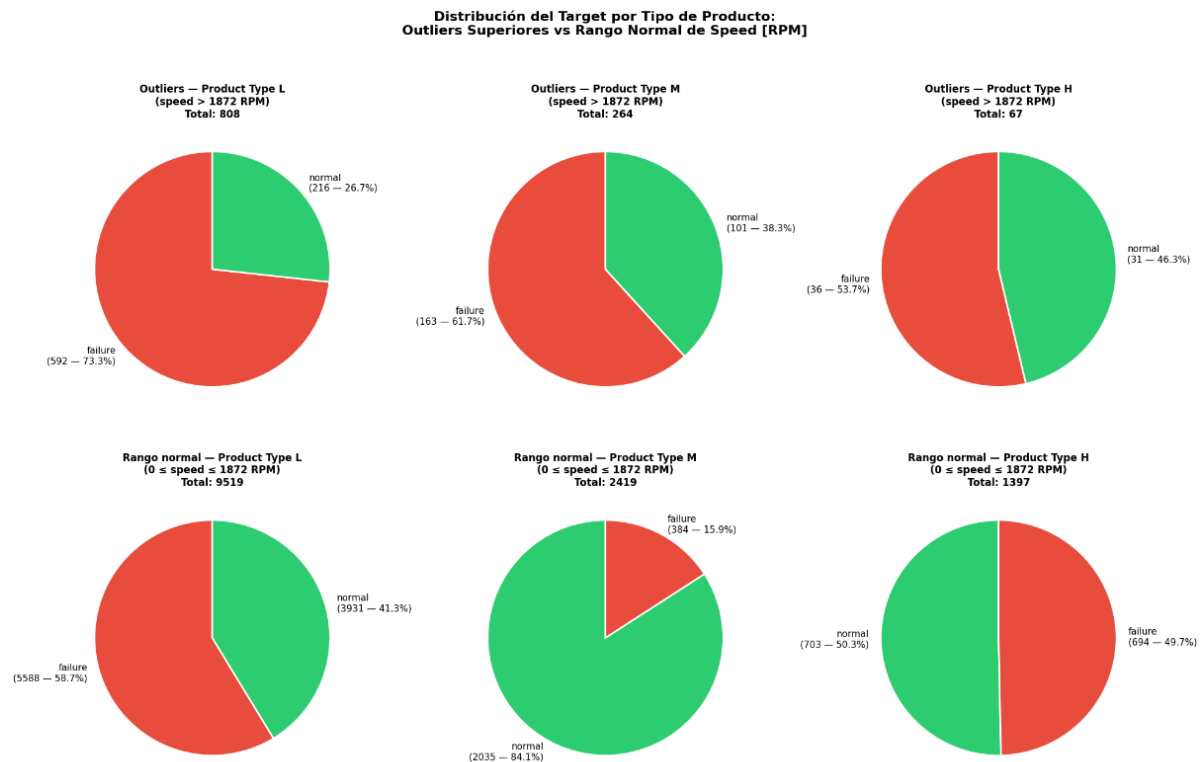


Figura 6: Distribución del target para Valores Normales vs Outliers en Speed para cada Tipo de Producto.

Al desagregar el análisis anterior por tipo de producto se observan comportamientos diferenciados ante las velocidades extremas:

- **Tipo L:** Consolida su posición como el componente más vulnerable. Incluso bajo condiciones normales de operación, exhibe una tasa de fallas básica muy elevada del **58.7%**, la cual escala hasta el **73.3%** al superar el umbral crítico de velocidad. Asimismo, es la variante que concentra la mayor cantidad de alertas, agrupando el **70.9%** del total de los *outliers* de velocidad (808 de 1,139 registros).
- **Tipo M:** Demuestra ser el componente mecánicamente más estable en condiciones nominales, registrando una tasa de fallas de apenas el **15.9%**. No obstante, es el tipo de producto más sensible al estrés cinético: ante velocidades atípicas, su probabilidad de fallo se dispara drásticamente al **61.7%**, representando el salto de criticidad más severo de todo el conjunto de datos.
- **Tipo H:** Exhibe un comportamiento prácticamente inmóvil ante el cambio de régimen, manteniendo una proporción de fallas cercana al 50% tanto en el rango normal (**49.7%**) como en el escenario de *outliers* (**53.7%**). Esta falta de sensibilidad ante una variable física tan crítica como la velocidad refuerza la hipótesis estadística de que esta

subpoblación de gama alta responde a un esquema de simulación o balanceo artificial previo.

En términos generales, el análisis conjunto de estos escenarios revela que el tipo H presenta un comportamiento global más robusto y seguro frente a las exigencias mecánicas en comparación con los tipos M y L (los cuales muestran vulnerabilidades críticas en rango normal o ante velocidades extremas, respectivamente). Esta tendencia decreciente a la susceptibilidad del fallo según la categoría aporta un leve indicio a favor de una estructura ordinal implícita en la variable `product_type`, sugiriendo que las etiquetas podrían representar diferentes niveles de resistencia o calidad de los componentes.

Análisis Comparativo de Distribuciones Numéricas por Tipo de Producto

Para comprender en profundidad las dinámicas operativas y los modos de falla específicos de cada categoría, se analizaron las variables físicas del dataset mediante gráficos de caja (boxplots):

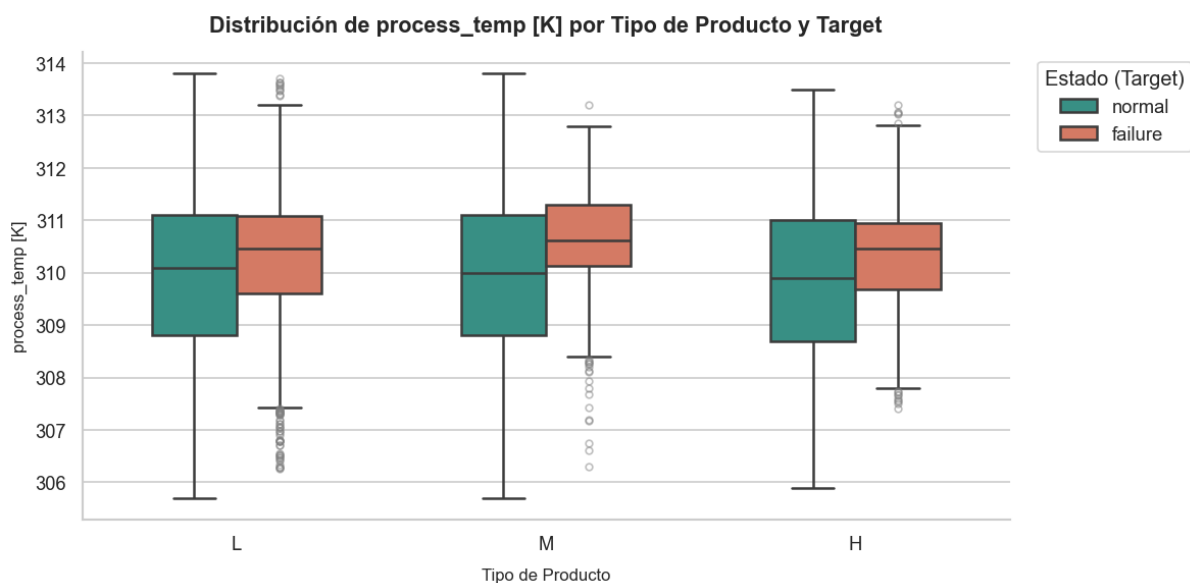


Figura 7: Diagrama de Cajas de la Temperatura del Proceso y del Aire por Tipo de Producto y Target

En las tres categorías (L, M y H), las instancias de failure se desplazan hacia temperaturas significativamente superiores en comparación con el estado normal (`air_temp` exhibe un comportamiento idéntico). Este fenómeno aporta evidencia sobre el hecho de que el sobrecalentamiento operativo actúa como un detonante crítico del fallo que se manifiesta de manera uniforme, independientemente del tipo de componente.

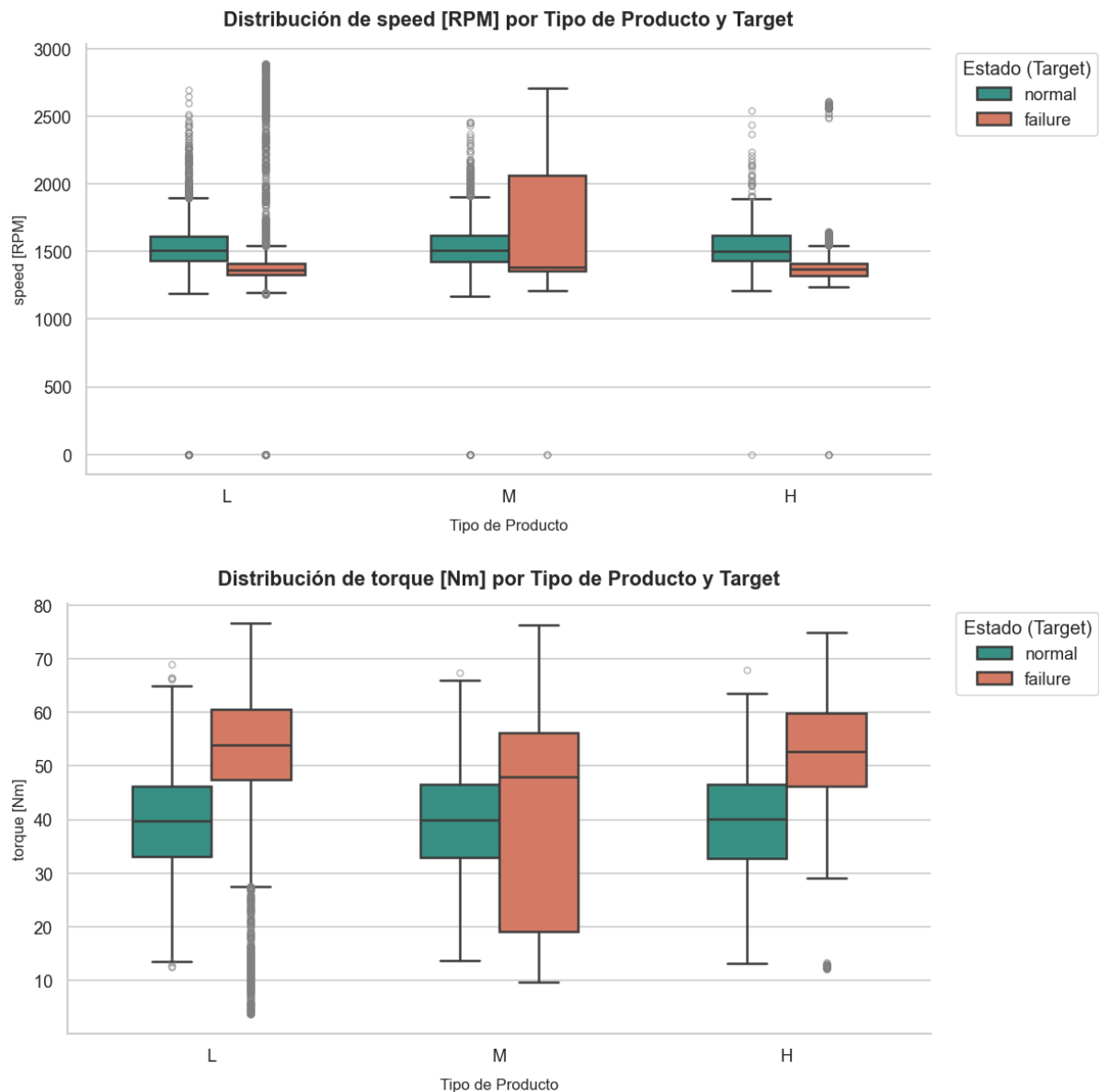


Figura 8: Diagrama de Cajas de la Velocidad y Torque por Tipo de Producto y Target

Las calidades L y H operan en un régimen estable (1500 RPM y torque de 40 Nm). Al fallar, sus registros se concentran en dos extremos claros: alta carga (baja velocidad con alto torque) o pérdida de resistencia (caída de torque por debajo de 15 Nm con altas RPM). Por el contrario, el Tipo M distribuye sus fallas a lo largo de rangos intermedios, registrando torques de entre 20 y 55 Nm y velocidades de 1300 a 2000 RPM, lo que indica que experimenta fallas dentro de su zona operativa común y no solo en los límites de esfuerzo.

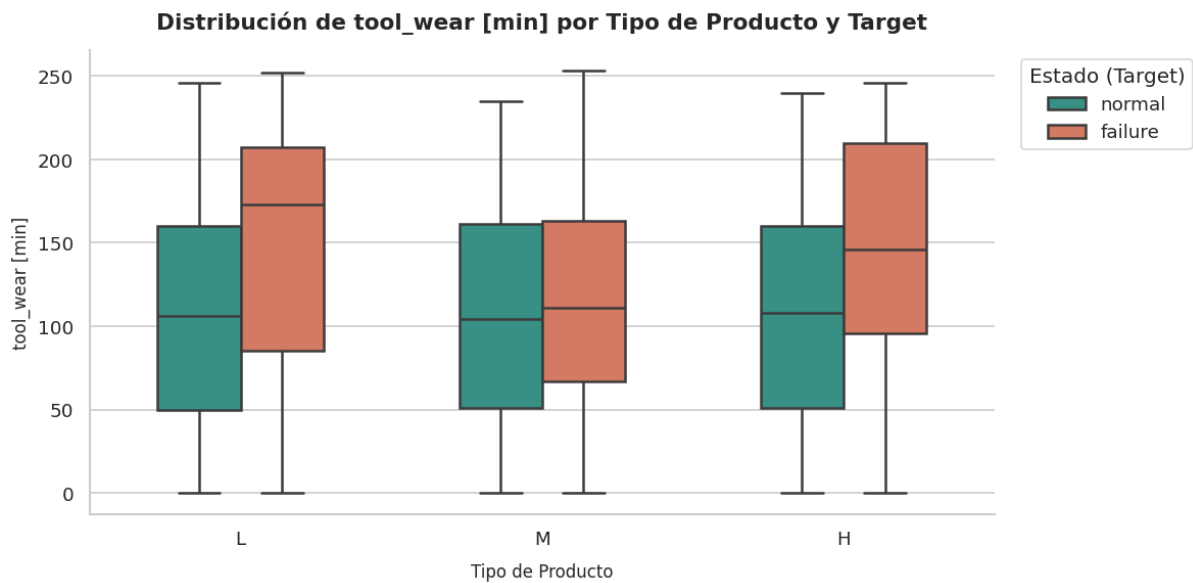


Figura 9: Diagrama de Cajas del uso de la herramienta por Tipo de Producto y Target

En este caso, los tipos L y H siguen un patrón esperado: a mayor desgaste acumulado, mayor probabilidad de falla, con medianas que superan los 150 y 170 minutos respectivamente en estado de falla, lo que confirma que estas herramientas fallan principalmente por desgaste el progresivo. Por otro lado, el tipo M, presenta una anomalía ya que sus medianas son casi idénticas en ambos estados (110 min), lo que indica que estas herramientas no fallan por desgaste acumulado, sino por factores operacionales como picos de torque o fluctuaciones de velocidad.

Análisis de Torque Crítico Inferior y Validación de Jerarquía

Outliers Inferiores de Torque (< 7.5 Nm)
Total: 107

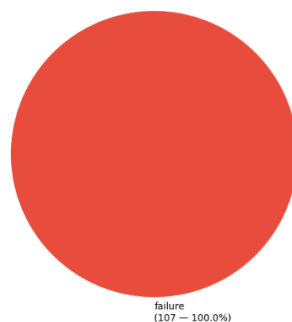


Figura 10: Distribución del Target en Outliers inferiores de Torque

El análisis del límite inferior de la variable torque [Nm] reveló un patrón predictivo absoluto de falla, detallado a continuación:

Umbral de Falla Absoluto: Se identificó un límite crítico determinista en los **12.6 Nm**. El 100% de los registros por debajo de este valor de torque (505 casos) corresponden a un estado de falla (failure). En ingeniería de procesos, una pérdida drástica y repentina de resistencia durante el mecanizado denota un colapso mecánico instantáneo, como la rotura de la herramienta de corte o la pérdida de sujeción de la pieza.

Comportamiento en Escenarios Extremos: Al desagregar este subconjunto de 505 fallas según el tipo de componente, la distribución de los casos resulta contundente:

- **Product Type L:** Concentra la gran mayoría con 363 fallas (**71.8%**).
- **Product Type M:** Presenta un comportamiento intermedio con 133 fallas (**26.3%**).
- **Product Type H:** Exhibe una resistencia superior con solo 9 fallas (**1.8%**).

Esta distribución decreciente de fallas por fragilidad representa un argumento más a favor de la hipótesis de la ordinalidad, sugiriendo que en situaciones de estrés crítico se mantiene una jerarquía de resistencia ($H > M > L$). Sin embargo, otras anomalías visualizadas contradicen este orden. La resolución de esta dualidad y la determinación definitiva sobre la naturaleza de la variable se establecerán más adelante mediante pruebas estadísticas más fehacientes.

Centrando el análisis en el tipo de producto H, cuyo comportamiento resultaba llamativo por su tasa de falla cercana al 50 %, los gráficos de cajas por target muestran que sus fallas se asocian a un patrón distinto: alto torque combinado con desgaste de herramienta acumulado, operando a velocidades moderadas. Es decir, este material no falla por velocidad sino por esfuerzo de corte elevado, lo que abrió la línea de análisis sobre la variable torque que se desarrolla a continuación.

El análisis de los outliers inferiores de torque arrojó el hallazgo más relevante de esta etapa. Por debajo de un umbral de **12,6 Nm**, el **100 % de los registros (505 casos) corresponden a fallas**, lo que convierte al torque bajo en un predictor prácticamente perfecto de la clase minoritaria.

	idx	parent_device_id	product_type	air_temp [K]	process_temp [K]	speed [RPM]	torque [Nm]	tool_wear [min]	target
	81	81	18566	L	301.76	311.05	2748	8.37	12
	124	124	18795	L	301.75	310.61	2669	11.83	122
	129	129	11162	L	303.48	312.67	2661	11.45	26
	146	146	18564	H	299.64	309.82	2584	12.52	89
	184	184	12538	M	299.59	309.56	2657	11.42	92


```

Distribución del target en torque < 12.6:
target
failure    505
Name: count, dtype: int64
product_type
L         363
M         133
H           9
Name: count, dtype: int64

```

Figura 11: Composición de fallas para torque bajo por tipo de producto

Al observar la composición de estas fallas de torque bajo por tipo de producto, se obtiene: L con 363 casos (71,8 %), M con 133 (26,3 %) y H con apenas 9 (1,8 %). Esto indica que los materiales L y M son mucho más propensos a este tipo de falla por colapso de torque, mientras que el material H prácticamente no lo presenta. Nuevamente, se trata de comportamientos cualitativamente distintos según el material, sin un orden que los jerarquice.

Relación Entre Variables

Análisis de correlación

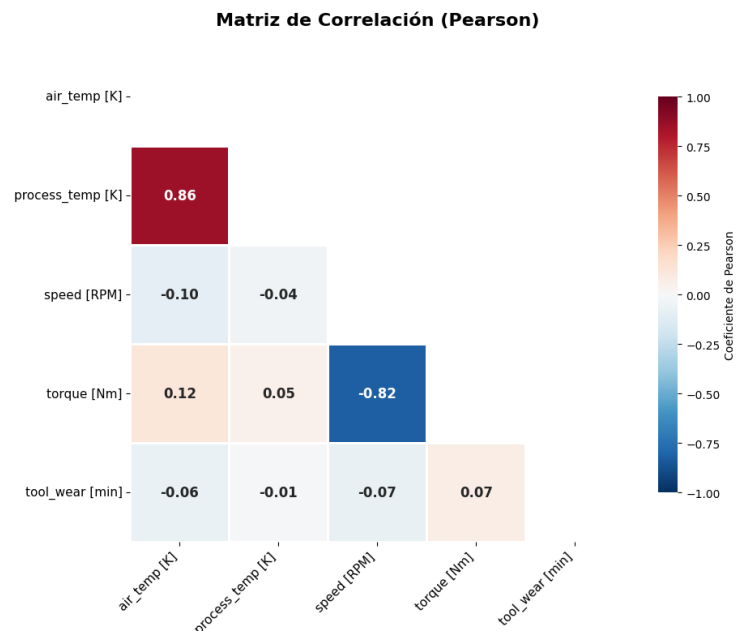


Figura 12: Matriz de correlación de Pearson.

Se calculó la matriz de correlación de Pearson entre las variables numéricas. Las relaciones más fuertes son speed y torque, con un coeficiente de $-0,82$ (a mayor velocidad, menor torque), y air_temp–process_temp, con $0,86$ (ambas temperaturas suben juntas). El resto de los pares presenta correlaciones débiles.

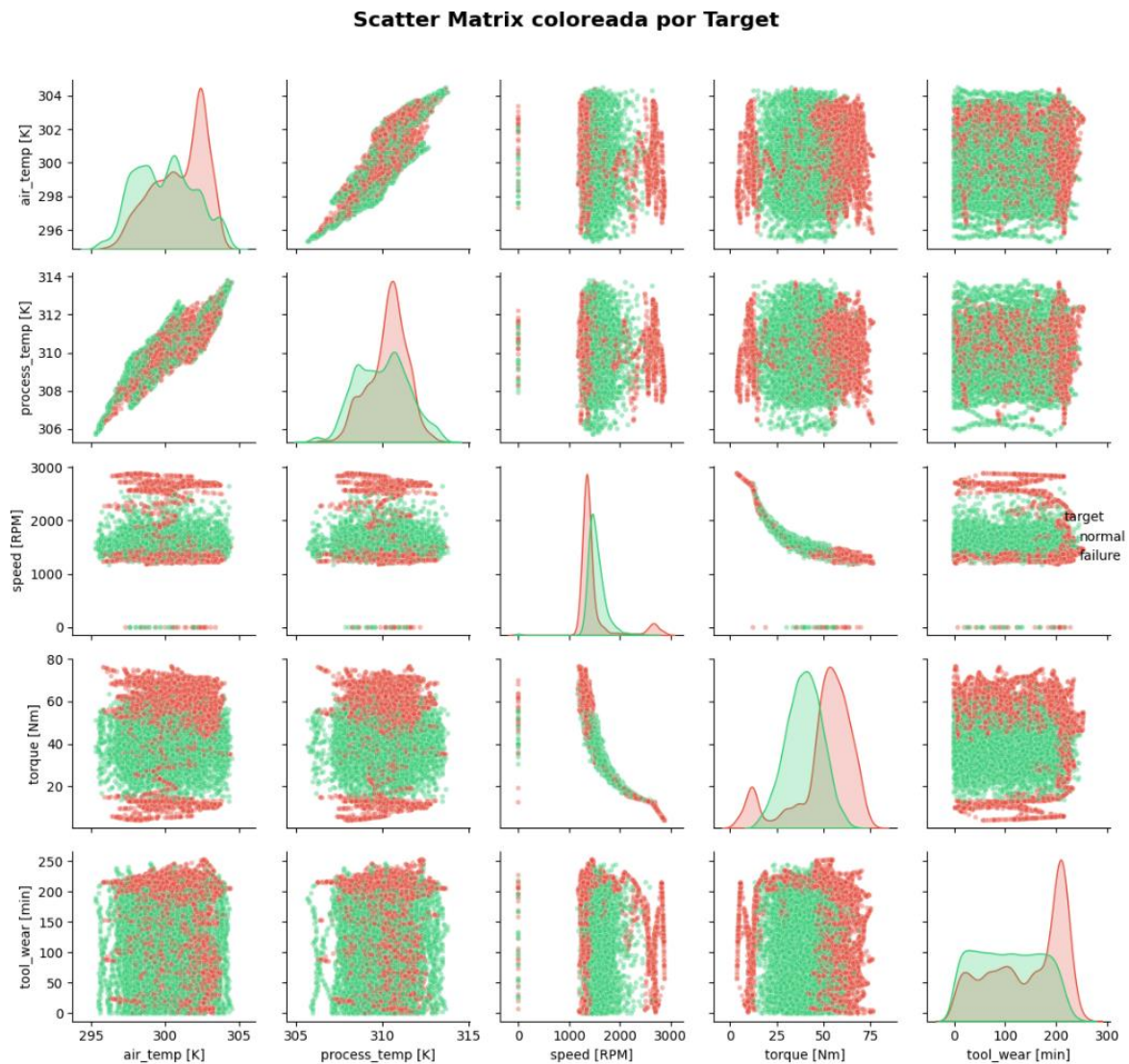


Figura 13: Matriz con gráficos de dispersión entre pares de variables y coloreado según target.

El diagrama de dispersión por pares, coloreado según el target, permite inspeccionar visualmente la forma de estas relaciones. Se observa que la relación entre speed y torque, aunque fuerte, no es estrictamente lineal, mientras que la de air_temp y process_temp sí presenta un patrón lineal claro.

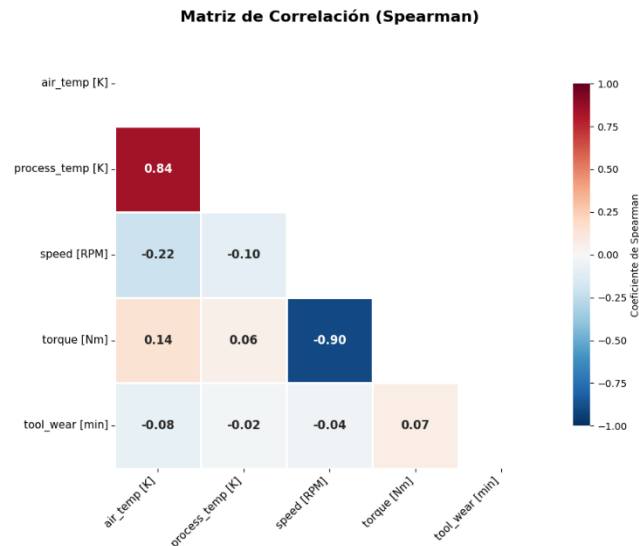


Figura 14: Matriz de correlación de Spearman.

Dado que Pearson solo captura relaciones lineales, se calculó también la correlación de Spearman, que mide asociaciones monótonas. La correlación de Spearman entre speed y torque resulta aún más fuerte que la de Pearson, lo que confirma que la relación entre ambas es monótona pero no lineal. Esta alta correlación entre las dos variables es relevante de cara al modelado, ya que indica redundancia parcial de información.

Si bien se consideró la posibilidad de eliminar speed o torque debido a la redundancia, luego de analizarlo en profundidad decidimos optar por mantener ambas columnas, ya que no valía la pena eliminarla ya que solo pasábamos a una dimensión menor (de 8 a 7).

Validación Estadística de la Naturaleza de product_type

Para fundamentar rigurosamente la estrategia de codificación, se contrastó la variable product_type frente al target utilizando dos enfoques estadísticos con objetivos independientes. Se aplicó el **Coefficiente de Correlación de Spearman** para evaluar la hipótesis de ordinalidad, ya que mide la fuerza de una relación monótona entre variables asignadas por rangos, determinando si un incremento en la calidad teórica ($L < M < H$) se traduce en una tendencia constante de cambio en las fallas. Por otro lado, se ejecutó un **Test Chi-Cuadrado** (χ^2) de Independencia para evaluar la hipótesis nominal, el cual no asume ningún orden ni distancia entre categorías, limitándose a comprobar si las frecuencias observadas de fallo difieren significativamente de lo esperado por puro azar entre los tres grupos.

Los resultados numéricos obtenidos en ambas pruebas fueron los siguientes:

- **Correlación de Spearman (Enfoque Ordinal):** ρ de -0.2349 con un p-valor de 3.61×10^{-181}
- **Chi-Cuadrado (Enfoque Nominal):** de 1331.5 con un p-value de 7.12×10^{-290}

La interpretación de estas métricas confirma que, aunque el *p-valor* de Spearman es estadísticamente significativo debido al gran volumen de la muestra, el coeficiente revela una asociación monótona débil. Esto se debe a la falta de linealidad global en el dataset completo, donde la tasa de fallos descende al pasar de L a M, pero repunta marcadamente al llegar a H. Este es un fenómeno que teorizamos que podría ser **consecuencia de una distorsión introducida por las técnicas de oversampling** aplicadas previamente sobre las zonas comunes. En contraposición, el test de Chi-cuadrado resulta concluyente, demostrando de manera contundente que las tasas de fallo reales varían de forma drástica entre las categorías. Esto valida una fuerte asociación nominal y asimétrica que responde a la naturaleza particular de cada material y no a una jerarquía marcada.

A partir de estas evidencias, queda descartado el uso de un mapeo ordinal ($L = 1$, $M = 2$, $H = 3$). Forzar distancias equivalentes introduciría relaciones geométricas falsas que perjudicarían el desempeño de modelos sensibles a la escala (regresión logística, KNN o redes neuronales). Para preservar el poder discriminante real de cada categoría sin imponer restricciones artificiales, se determina tratar a **product_type** como una **variable nominal** mediante la aplicación de **One-Hot Encoding**, permitiendo que los algoritmos evalúen el impacto específico de cada tipo de componente de manera independiente durante el modelado y entrenamiento.

Preprocesamiento de los datos

Una vez finalizado el análisis exploratorio, se continuó con la preparación del conjunto de datos para su uso en modelos de aprendizaje automático. El preprocesamiento se organizó en las siguientes etapas: eliminación de columnas irrelevantes, codificación de variables categóricas, división en entrenamiento y prueba, tratamiento de valores inválidos y faltantes, y normalización.

Eliminación de columnas irrelevantes

Se eliminaron las columnas `idx` y `parent_device_id`, ya que se trata de identificadores que no aportan información predictiva: el primero es un índice de registro y el segundo

identifica el equipo de origen, sin relación causal con la ocurrencia de fallas. Mantenerlas solo agregaría ruido y dimensionalidad innecesaria.

Codificación de variables categóricas

	air_temp [K]	process_temp [K]	speed [RPM]	torque [Nm]	tool_wear [min]	target	product_type_H	product_type_L	product_type_M
1448	NaN	312.40	1404	61.50	127	1	False	True	False
1449	NaN	311.51	1371	47.22	60	1	False	True	False
1450	NaN	309.51	1337	63.61	206	1	False	True	False
3901	NaN	309.82	1981	20.50	203	1	False	True	False
3902	NaN	311.00	1428	47.90	174	0	False	True	False
3903	NaN	312.34	2687	10.48	67	1	False	False	True
3904	NaN	311.50	1663	33.60	38	0	False	True	False

Tabla 4: Resultado de codificación one-hot para product_type.

La variable product_type se codificó mediante One-Hot Encoding, generando tres columnas binarias independientes (product_type_L, product_type_M, product_type_H). Esta decisión se fundamentó en la sección de Validación Estadística mediante pruebas de Spearman y Chi-Cuadrado.

División del conjunto de datos

El conjunto de datos se dividió en una proporción de 80% para entrenamiento y 20% para prueba antes de realizar cualquier imputación o escalado, para de esta manera evitar la fuga de información (data leakage) desde el conjunto de prueba:

- **Entrenamiento (Train):** 11,616 muestras (51.5% failure / 48.5% normal).
- **Prueba (Test):** 2,905 muestras (51.5% failure / 48.5% normal).

La partición se realizó utilizando estratificación sobre la variable objetivo (stratify=y) para garantizar que ambos subconjuntos conserven exactamente la misma proporción de clases, asegurando una validación simétrica y libre de sesgos en la etapa de modelado.

Tratamiento de valores inválidos y faltantes

Durante el análisis exploratorio se identificaron dos problemas distintos en los datos:

- **Valores inválidos en speed:** la variable presentaba registros con valor -1, que no corresponden a mediciones reales sino a datos faltantes codificados con un valor centinela o a errores de medición de los sensores.

	air_temp [K]	process_temp [K]	speed [RPM]	torque [Nm]	tool_wear [min]	product_type_H	product_type_L	product_type_M
8668	298.90	309.60	-1	33.70	195	False	True	False
8677	303.06	311.50	-1	49.02	88	False	True	False
8687	300.30	310.40	-1	54.40	52	False	True	False
58	300.70	311.35	-1	19.05	227	False	False	True
51	300.70	310.60	-1	30.30	96	False	True	False
8680	298.72	309.23	-1	61.47	216	False	True	False

Tabla 5: Ejemplos de algunos registros con valor -1 en speed [RPM].

- **Valores faltantes en air_temp:** 40 registros con valores ausentes (NaN) reales.

Ambos casos se trataron por separado y siempre tomando el conjunto de entrenamiento como referencia.

```
Cantidad de valores nulos en el conjunto de entrenamiento:
air_temp [K]      35
process_temp [K]   0
speed [RPM]       0
torque [Nm]       0
tool_wear [min]   0
product_type_H    0
product_type_L    0
product_type_M    0
dtype: int64
Cantidad de valores nulos en el conjunto de prueba:
air_temp [K]      5
process_temp [K]   0
speed [RPM]       0
torque [Nm]       0
tool_wear [min]   0
product_type_H    0
product_type_L    0
product_type_M    0
dtype: int64
```

Figura 15: Cantidad de valores nulos de air_temp antes de la imputación.

Para los valores -1 de speed, se optó por una imputación basada en la relación física entre variables. Dado que en el análisis exploratorio se observó una fuerte correlación entre speed y torque, cada valor inválido se imputó con el promedio de la velocidad de los 5 registros con torque más parecido. Para evitar fugas de información, los vecinos siempre se buscaron dentro del conjunto de entrenamiento, tanto al imputar el train como el test. Esta imputación se realizó sobre los valores en su escala original, previo a la normalización.

Tras la imputación se verificó que no quedaran valores -1 en ninguno de los dos subconjuntos.

```
Valores -1 en X_train['speed [RPM]']: 0
Valores -1 en X_test['speed [RPM]']: 0
```

Figura 16: Cantidad de registros con -1 en speed luego de la imputación.

Para los 40 valores faltantes de `air_temp`, se utilizó el imputador *KNNImputer* de *scikit-learn* con 5 vecinos, ajustado únicamente sobre el conjunto de entrenamiento y luego aplicado al de prueba. Si bien creemos que para un conjunto de datos de estas dimensiones una imputación simple por media o mediana habría sido suficiente, se optó por KNN con fines didácticos, ya que aprovecha la similitud entre registros en el espacio de las demás variables para producir una estimación más informada.

```
Cantidad de valores nulos en el conjunto de entrenamiento:
air_temp [K]      0
process_temp [K]  0
speed [RPM]       0
torque [Nm]       0
tool_wear [min]   0
product_type_H    0
product_type_L    0
product_type_M    0
dtype: int64
Cantidad de valores nulos en el conjunto de prueba:
air_temp [K]      0
process_temp [K]  0
speed [RPM]       0
torque [Nm]       0
tool_wear [min]   0
product_type_H    0
product_type_L    0
product_type_M    0
dtype: int64
```

Figura 17: Cantidad de valores nulos de `air_temp` luego de la imputación.

Finalizada esta etapa se confirmó que el conjunto no contenía valores faltantes ni inválidos.

```
Estadísticas del train después del escalado:
air_temp [K]  process_temp [K]  speed [RPM]  torque [Nm]  \
count      11581.00      11616.00      11616.00      11616.00
mean         -0.00         -0.00         -0.00         -0.00
std           1.00           1.00           1.00           1.00
min          -2.67          -3.38          -1.16          -2.90
25%          -0.81          -0.73          -0.54          -0.63
50%           0.06           0.14          -0.31           0.13
75%           0.89           0.69           0.14           0.70
max           2.07           2.75           4.56           2.22

tool_wear [min]
count      11616.00
mean         -0.00
std           1.00
min          -1.79
25%          -0.86
50%           0.05
75%           0.96
max           1.82
```

Figura 18: Cantidad de valores nulos de `air_temp` luego de la imputación.

Normalización

Las variables numéricas presentaban escalas muy dispares: por ejemplo, `air_temp` se mueve en el rango 296–303, mientras que `speed` oscila entre 1200 y 1700 y `tool_wear` entre 0

y 250. Estas diferencias de magnitud afectan especialmente a los modelos basados en distancia, como KNN, que de otro modo estarían dominados por las variables de mayor rango. Por ello se aplicó estandarización (StandardScaler), que centra cada variable en media 0 y desvío 1. El escalador se ajustó (fit) exclusivamente sobre el conjunto de entrenamiento y luego se aplicó (transform) tanto al de entrenamiento como al de prueba, manteniendo la coherencia con el resto del pipeline y evitando fugas de información.

Balance del conjunto de datos

El enunciado solicita considerar el balance de clases. Como se observó en el análisis exploratorio, el conjunto está prácticamente equilibrado ($\approx 51,5\%$ fallas y $\approx 48,5\%$ normales), un desbalance leve que no justifica la aplicación de técnicas de remuestreo. En su lugar, se optó por optimizar la métrica F1 en la búsqueda de hiperparámetros y por evaluar la opción *class_weight='balanced'* en los modelos basados en árboles.

Entrenamiento y evaluación de modelos

Para resolver el problema de clasificación binaria (predecir la presencia de falla) se entrenaron cuatro modelos de distinta naturaleza, de modo de cubrir familias de algoritmos diferentes: uno basado en distancia (KNN), uno probabilístico (Naive Bayes), uno basado en reglas (Árbol de Decisión) y uno ensamblado (Random Forest).

Metodología común

Todos los modelos siguieron el mismo procedimiento. Para la búsqueda de hiperparámetros se utilizó RandomizedSearchCV, que evalúa un subconjunto aleatorio del espacio de configuraciones previamente definido mediante validación cruzada de 5 particiones (cv=5). Se eligió la búsqueda aleatoria sobre la exhaustiva (GridSearchCV) por su eficiencia: en el caso del Árbol de Decisión, por ejemplo, el grid completo contemplaba 8.208 combinaciones de hiperparámetros posibles, mientras que la búsqueda aleatoria explora una muestra representativa a una fracción del costo computacional.

Como métrica de optimización se utilizó **F1-score** en lugar de accuracy. En un contexto de mantenimiento predictivo el objetivo es detectar correctamente las fallas (clase positiva), y F1 equilibra precision y recall, penalizando tanto los falsos negativos (fallas no detectadas, que en la práctica implican paradas no previstas) como los falsos positivos (mantenimientos

innecesarios). Cada modelo se evaluó finalmente sobre el conjunto de prueba mediante accuracy, precision, recall, F1 y la matriz de confusión.

KNN

```

--- Resultados para KNN ---
Accuracy: 0.9601
Precision: 0.9390
Recall: 0.9866
F1-Score: 0.9622

Reporte de Clasificación:

```

	precision	recall	f1-score	support
0	0.98	0.93	0.96	1408
1	0.94	0.99	0.96	1497
accuracy			0.96	2905
macro avg	0.96	0.96	0.96	2905
weighted avg	0.96	0.96	0.96	2905

Figura 19: Reporte de clasificación de KNN

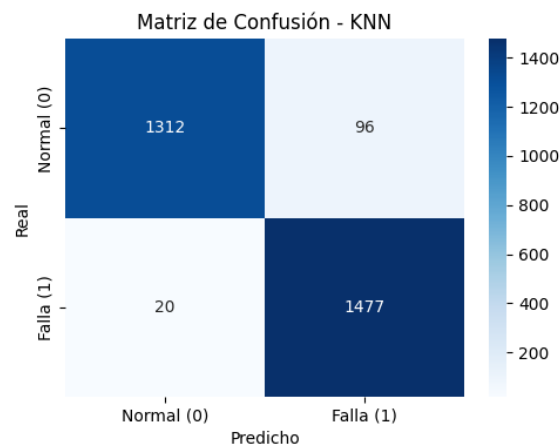


Figura 20: Matriz de confusión de KNN

Mejores hiperparámetros KNN: `{'weights': 'uniform', 'n_neighbors': 1, 'metric': 'minkowski'}`

El mejor modelo encontrado utiliza `n_neighbors=1`, pesos uniformes y métrica Minkowski. Sobre el conjunto de prueba alcanzó un F1 de 0,962, con un recall de 0,987 para la clase falla, es decir, detecta casi todas las fallas reales. Cabe señalar que `n_neighbors=1` es un valor muy bajo que normalmente indica riesgo de sobreajuste; en este caso funciona bien gracias a que el escalado previo y la fuerte separabilidad de las clases (observada en el análisis exploratorio) hacen que el vecino más cercano sea casi siempre informativo.

Naive Bayes

```

--- Resultados para Naive Bayes ---
Accuracy: 0.8437
Precision: 0.8337
Recall: 0.8704
F1-Score: 0.8516

Reporte de Clasificación:

```

	precision	recall	f1-score	support
0	0.86	0.82	0.83	1408
1	0.83	0.87	0.85	1497
accuracy			0.84	2905
macro avg	0.84	0.84	0.84	2905
weighted avg	0.84	0.84	0.84	2905

Figura 21: Reporte de clasificación de Naive Bayes

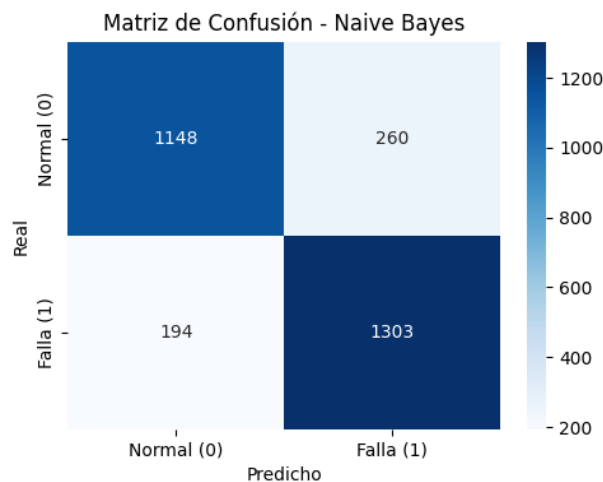


Figura 22: Matriz de confusión de Naive Bayes

Mejores hiperparámetros Naive Bayes: `{'var_smoothing': np.float64(0.3511191734215131)}`

El modelo gaussiano, con `var_smoothing` optimizado, fue el de menor desempeño ($F1 = 0,852$). Este resultado es esperable: Naive Bayes asume independencia condicional entre las variables, supuesto que se viola claramente dada la fuerte correlación detectada entre `speed-torque` y `air_temp-process_temp`. Aun así, se incluyó como modelo de referencia por su simplicidad y bajo costo.

Árbol de Decisión

```

--- Resultados para Decision Trees ---
Accuracy: 0.9528
Precision: 0.9462
Recall: 0.9633
F1-Score: 0.9547

Reporte de Clasificación:

```

	precision	recall	f1-score	support
0	0.96	0.94	0.95	1408
1	0.95	0.96	0.95	1497
accuracy			0.95	2905
macro avg	0.95	0.95	0.95	2905
weighted avg	0.95	0.95	0.95	2905

Figura 23: Reporte de clasificación de Árbol de Decisión.

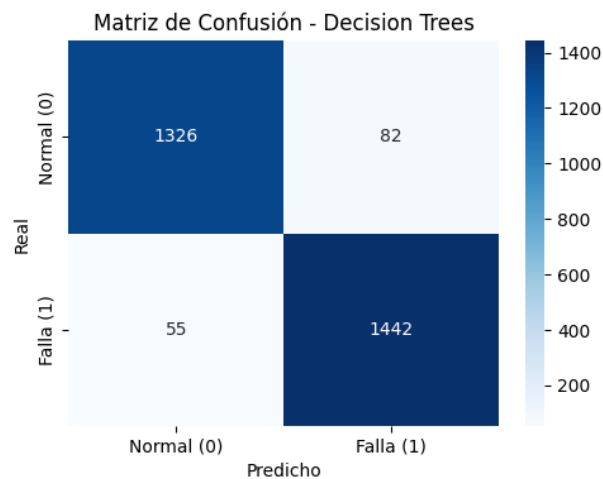


Figura 24: Matriz de confusión de Árbol de Decisión.

Mejores hiperparámetros Decision Tree: {'min_samples_split': 4, 'min_samples_leaf': 1, 'max_depth': 15, 'criterion': 'entropy', 'class_weight': None}

El mejor árbol utiliza criterio de entropía, profundidad máxima de 15 y `min_samples_split=4`. Alcanzó un F1 de 0,955, muy competitivo. Los modelos basados en reglas se benefician especialmente de los patrones identificados en el análisis exploratorio, como el umbral de torque por debajo del cual todos los registros son fallas, que el árbol puede capturar directamente mediante un corte.

Random Forest (modelo ensamblado)

```

--- Resultados para Random Forest ---
Accuracy:  0.9711
Precision: 0.9651
Recall:    0.9793
F1-Score:  0.9721

Reporte de Clasificación:

```

	precision	recall	f1-score	support
0	0.98	0.96	0.97	1408
1	0.97	0.98	0.97	1497
accuracy			0.97	2905
macro avg	0.97	0.97	0.97	2905
weighted avg	0.97	0.97	0.97	2905

Figura 25: Reporte de clasificación de Random Forest

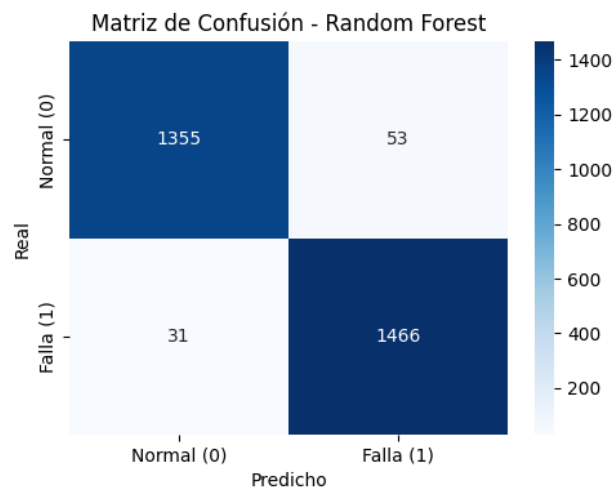


Figura 26: Matriz de confusión de Random Forest.

Mejores hiperparámetros Random Forest: `{'n_estimators': 300, 'min_samples_split': 2, 'min_samples_leaf': 2, 'max_depth': 30, 'criterion': 'entropy', 'class_weight': None, 'bootstrap': False}`

Como modelo ensamblado se entrenó un Random Forest, que combina múltiples árboles de decisión para reducir la varianza y mejorar la generalización. La mejor configuración utiliza 300 estimadores, criterio de entropía, profundidad máxima de 30 y `bootstrap=False`. Obtuvo el mejor desempeño de todos los modelos, con un F1 de 0,972 y métricas equilibradas entre ambas clases, confirmando la ventaja del enfoque ensamblado sobre un árbol individual.

Comparación de Modelos

	Accuracy	Precision	Recall	F1-Score
Modelo				
Random Forest	0.9711	0.9651	0.9793	0.9721
KNN	0.9601	0.9390	0.9866	0.9622
Decision Tree	0.9528	0.9462	0.9633	0.9547
Naive Bayes	0.8437	0.8337	0.8704	0.8516

Tabla 6: Resultados numéricos de las métricas de desempeño de los modelos utilizados.

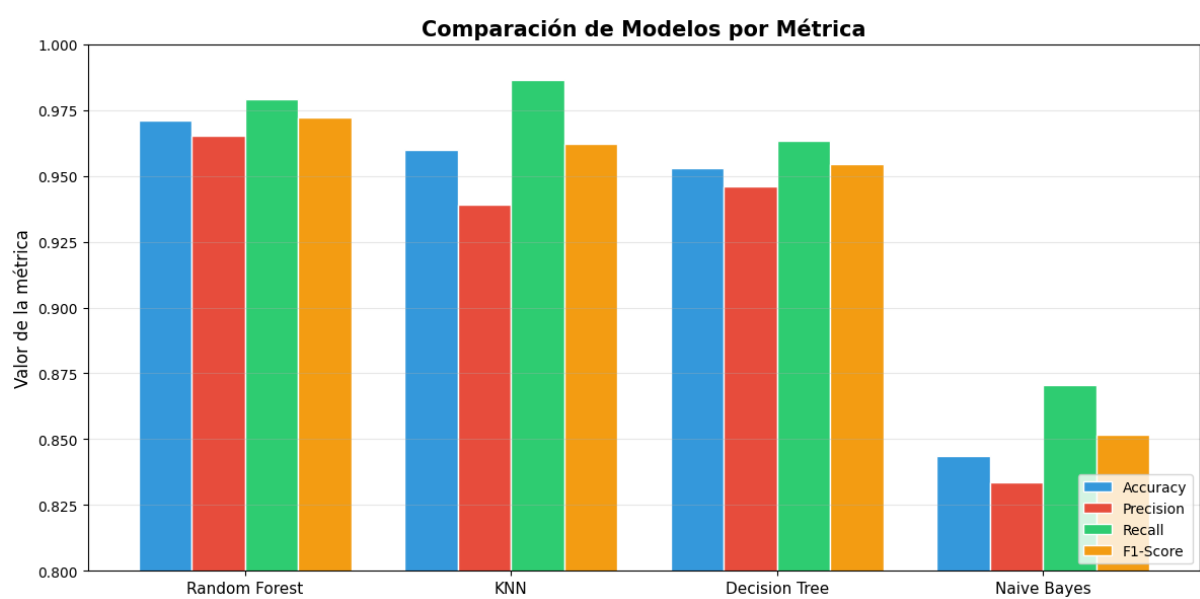


Figura 27: Comparación de modelos por métrica.

La tabla y el gráfico comparativo resumen el desempeño de los cuatro modelos sobre el conjunto de prueba. Random Forest obtuvo el mejor resultado global ($F1 = 0,972$), seguido de cerca por KNN ($0,962$) y Árbol de Decisión ($0,955$), los tres superan el $0,95$, lo que indica que el problema resultó altamente separable. Naive Bayes quedó claramente por debajo ($F1 = 0,852$), consecuencia de su supuesto de independencia entre variables, incompatible con las correlaciones detectadas en los datos.

El alto rendimiento general se explica en buena medida por los patrones identificados durante el análisis exploratorio, en particularidad de generalización y su desempeño equilibrado entre ambas clases, Random Forest se selecciona como el modelo final recomendado para este problema de mantenimiento predictivo.

Curva ROC y AUC

AUC	
Modelo	
Random Forest	0.9970
Decision Tree	0.9587
KNN	0.9586
Naive Bayes	0.9155

Tabla 7: Resultados numéricos de AUC para los modelos utilizados

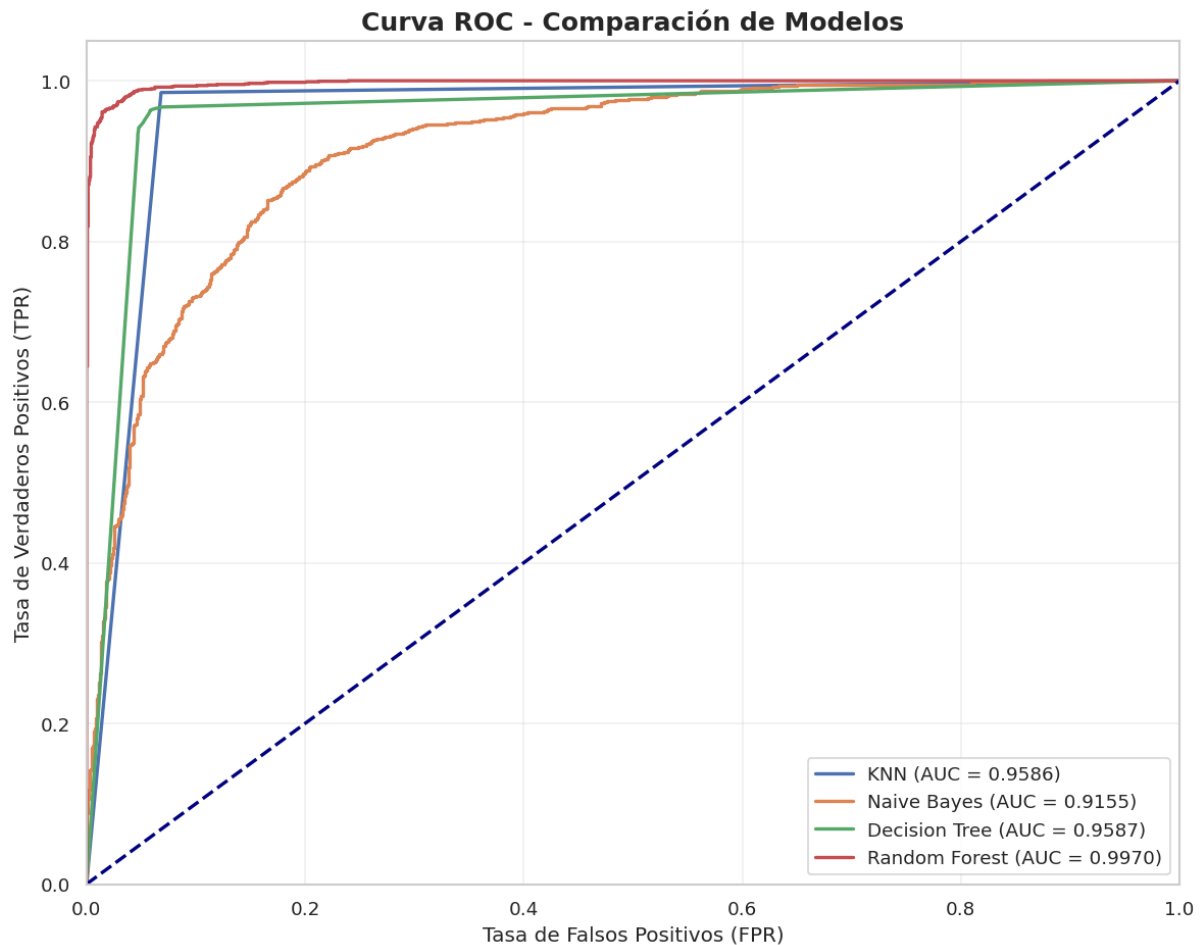


Figura 28: Curva ROC y AUC por tipo de modelo empleado.

La curva ROC permite evaluar qué tan bien distingue cada modelo entre una falla y un funcionamiento normal. El AUC resume este desempeño en un único valor: cuanto más se acerca a 1, mayor es la capacidad predictiva del modelo.

En este caso, Random Forest es el que mejor desempeño tiene con un AUC de 0.997, prácticamente perfecto, lo que refleja una alta capacidad para detectar fallas reales mientras mantiene los falsos positivos al mínimo.

También, KNN y el Árbol de Decisión se sitúan en un nivel muy similar, con AUC de 0.9586 y 0.9587 respectivamente, ambos ofrecen un desempeño sólido, aunque el Árbol de Decisión muestra una curva más escalonada producto de sus reglas de partición rígidas, mientras que KNN se beneficia de la estandarización previa de las variables. Por último, Naive Bayes cierra el ranking con un AUC de 0.915, el más bajo del grupo, lo que indica una menor capacidad para separar ambas clases con precisión.

En términos generales, los cuatro modelos superan el umbral de 0.91, lo que habla de un conjunto de clasificadores competentes. Sin embargo, Random Forest se posiciona como la opción más confiable para un entorno de predicción de fallas donde los errores tienen consecuencias operativas reales.

Tiempos de cómputo

	Tiempo Entrenamiento (s)	Tiempo Predicción (s)
Modelo		
KNN	0.0134	0.0188
Naive Bayes	0.0038	0.0011
Decision Tree	0.0415	0.0015
Random Forest	3.6921	0.0761

Tabla 8: Resultados numéricos de tiempo de cómputo para los modelos utilizados

El análisis de tiempos de cómputo revela diferencias relevantes entre modelos, son las siguientes:

- Random Forest es el más lento en entrenamiento con 3,69 segundos, pero en un entorno de mantenimiento predictivo donde el reentrenamiento ocurre de forma periódica, este costo es perfectamente asumible considerando la precisión que ofrece. Su tiempo de predicción de 0,076 segundos tampoco representa una limitación práctica.
- Naive Bayes destaca por su velocidad en ambas fases: entrena en apenas 0,004 segundos y predice en 0,001 segundos, lo que lo convierte en una opción atractiva para sistemas con recursos computacionales limitados.
- El Árbol de Decisión sigue una lógica similar a los anteriores, siendo el más veloz en predicción junto a Naive Bayes, dado que clasificar un nuevo caso se reduce a recorrer un conjunto de reglas simples, una ventaja real si el sistema requiere respuesta en milisegundos.
- KNN presenta un comportamiento particular: entrena rápido porque simplemente almacena los datos en memoria, pero predice más lento que Naive Bayes porque, al momento de clasificar, debe calcular la distancia entre el nuevo punto y todos los registros existentes.

En definitiva, si la prioridad es la precisión, Random Forest es la elección más conveniente. Si el sistema demanda respuesta casi inmediata u opera con recursos acotados, el Árbol de Decisión o Naive Bayes ofrecen una alternativa eficiente y suficientemente competente.

Las diferencias de tiempo pueden apreciarse en los siguientes gráficos, presentados tanto en escala real como logarítmica para facilitar la comparación entre modelos.

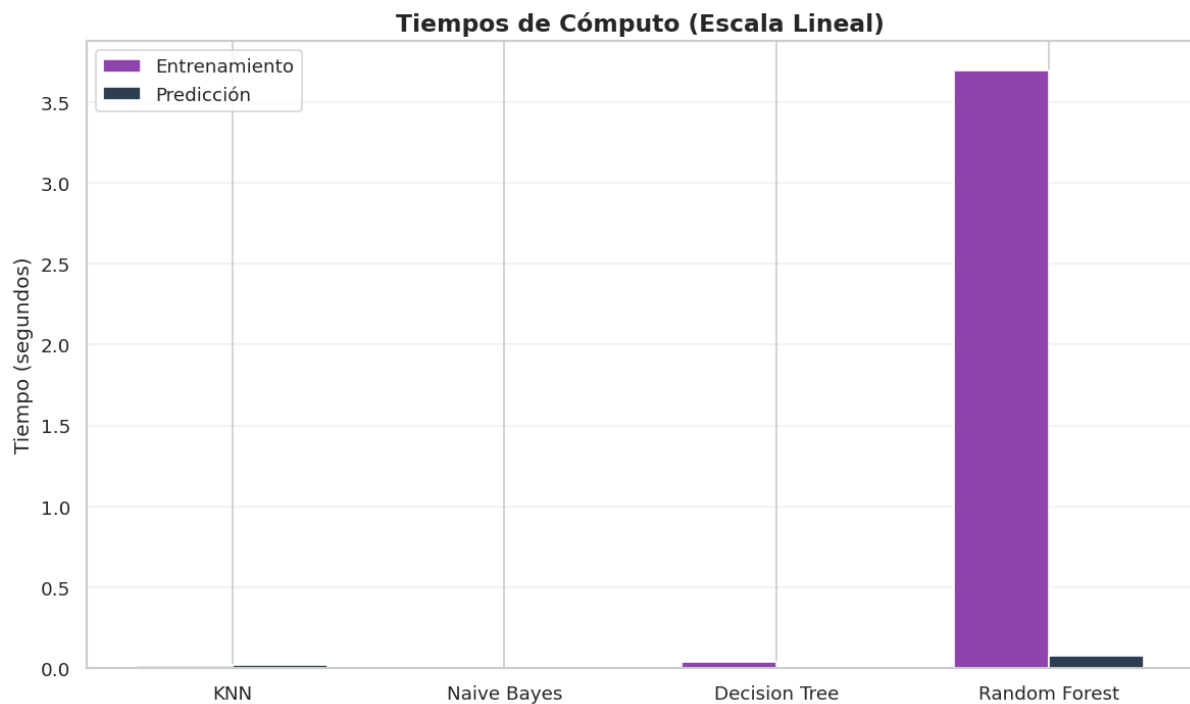


Figura 29: Tiempos de cómputo en escala lineal.

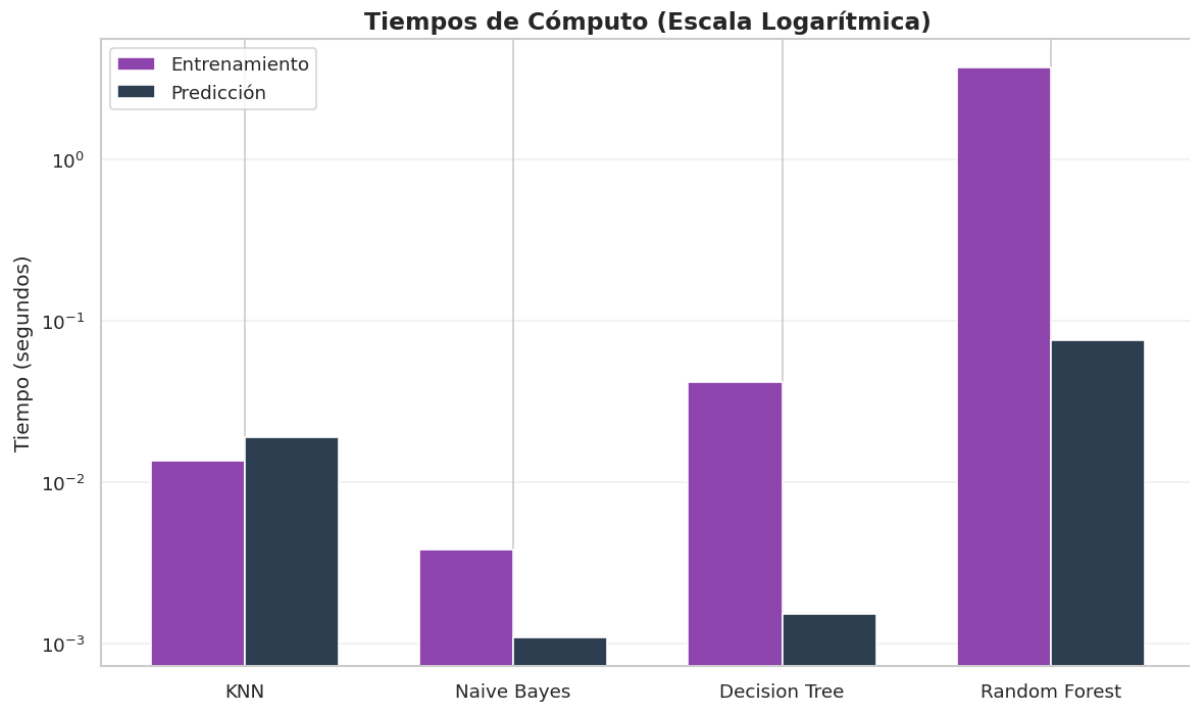


Figura 30: Tiempos de cómputo en escala logarítmica.

Conclusión

El desarrollo del presente trabajo permitió abordar de manera integral un problema de clasificación binaria orientado al mantenimiento predictivo industrial, recorriendo todas las etapas habituales de un proyecto de aprendizaje automático: análisis exploratorio, preprocesamiento, entrenamiento y evaluación de modelos. A lo largo del proceso, el análisis exploratorio se reveló como una etapa fundamental, ya que no solo permitió comprender la estructura y la calidad del conjunto de datos, sino que también aportó hallazgos sustantivos para la posterior toma de decisiones de modelado. Entre ellos, se destacan la identificación de valores inválidos en speed [RPM], la presencia de valores faltantes en air_temp [K], el umbral crítico de torque por debajo del cual el 100% de los registros corresponden a fallas, y la validación estadística que permitió determinar la naturaleza nominal de la variable product_type.

En lo que respecta al desempeño de los modelos, los resultados obtenidos confirman que el problema planteado resultó altamente separable, ya que tres de los cuatro algoritmos evaluados superaron el umbral de 0,95 en F1-score. Random Forest se consolidó como el modelo de mejor desempeño global (F1 = 0,972; AUC = 0,997), seguido de cerca por KNN y el Árbol de Decisión, mientras que Naive Bayes quedó claramente por debajo debido a que su

supuesto de independencia condicional resulta incompatible con las fuertes correlaciones detectadas entre speed–torque y air_temp–process_temp. Al mismo tiempo, el análisis de tiempos de cómputo permitió contrastar la precisión obtenida frente al costo computacional asociado, evidenciando que la elección del modelo final depende no solo de las métricas de calidad predictiva, sino también de las restricciones operativas del entorno de despliegue.

En conclusión, Random Forest es seleccionado como el modelo recomendado para este problema de mantenimiento predictivo, dado que combina una alta capacidad de detección de fallas reales con una baja tasa de falsos positivos, lo que en un contexto industrial se traduce en una reducción efectiva tanto de paradas no planificadas como de mantenimientos innecesarios. Más allá del resultado puntual, el trabajo permitió consolidar la importancia de un análisis exploratorio riguroso como base de cualquier solución de aprendizaje automático, así como la necesidad de fundamentar cada decisión de preprocesamiento y modelado en evidencia empírica y estadística sólida.